

Student: Gourik Narendra Patil
Date: 14 June 2026
Final Outcome Tier: Strong Pass

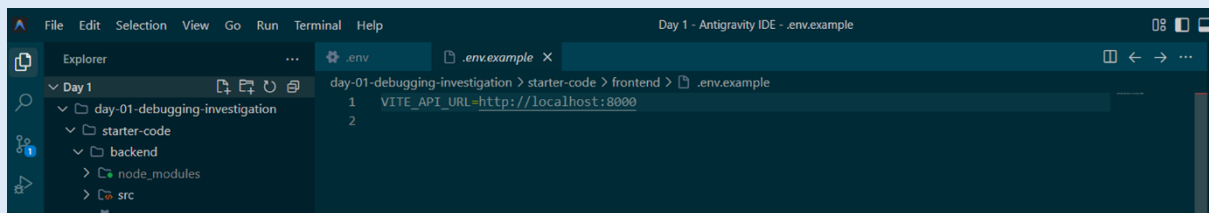
Date: 14 June 2026

Final Outcome Tier: Strong Pass

Debugging Discipline (Day 1)

The Action: Instead of assuming the backend code was flawed, network requests were traced to reveal a port configuration mismatch (frontend targeting 8000, backend running on 5000).

The Takeaway: Runtime evidence must always supersede code assumptions.

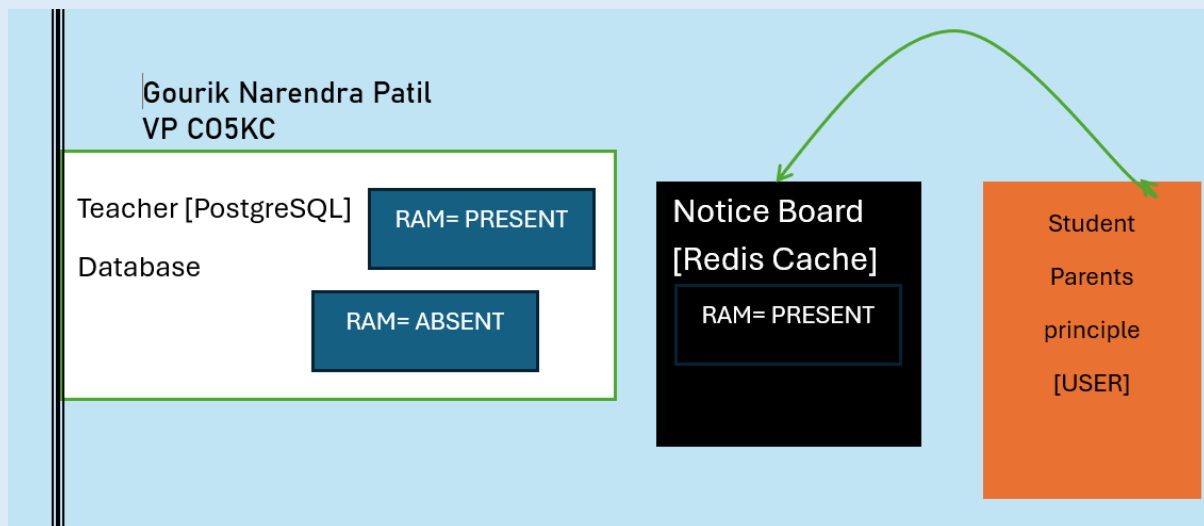


Systems Reasoning (Day 2)

The Incident: UI status mismatches and duplicate shortlist actions.

The Action: Traced the issue back to a Redis cache invalidation failure (18.5% mismatch rate). Prioritized disabling the cache to restore data consistency, accepting the tradeoff of a slower dashboard.

The Takeaway: Downstream symptoms (UI bugs) must be separated from origin failures (cache poisoning). Data correctness is more critical than speed.



Readiness Verification (Day 3)

The Incident: Evaluating deployment readiness.

The Action: Discovered a wrong `.env` file name breaking JWT authentication and a hardcoded constraint in `routes.js` blocking candidates. Blocked the launch until the full end-to-end recruiter workflow was verified.

The Takeaway: "Ready" means the critical path is proven, not just that a login was successful.

```
> node src/server.js
[INFO] Starting HireSignal API
[INFO] NODE_ENV=development
[INFO] Backend running on port 5000
[ERROR] POST /api/login 500 Internal Server Error
Error: secretOrPrivateKey must have a value
    at module.exports [as sign] (G:\Challenge\day 3\day 3 do\day-03-beyond-localhost-d3-c\starter-code\backend\node_modules\jsonwebtoken\sign.js:111:20)
    at login (file:///G:/Challenge/day%203/day%203%20do/day-03-beyond-localhost-d3-c/starter-code/backend/src/auth.js:16:21)
    at Layer.handle [as handle_request] (G:\Challenge\day 3\day 3 do\day-03-beyond-localhost-d3-c\starter-code\backend\node_modules\express\lib\router\layer.js:95:5)
```

```
62
63     if (candidate.id === "c_104") {
64         warn("Resume text quality below analysis threshold for candidate_id=c_104");
65         error("Resume analysis failed: malformed_or_missing_resume_content");
66         warn("No graceful fallback message configured for candidate_id=c_104");
67         return res.status(500).json({
68             message: "Resume content is not analysis-ready for this candidate"
69         });
70     }
```

Ambiguity Ownership (Day 4)

The Incident: Undefined notification behavior during manual shortlisting.

The Action: Designed a minimal safe v1. Delivered the requested recruiter control but explicitly blocked automated notifications to prevent accidental communications.

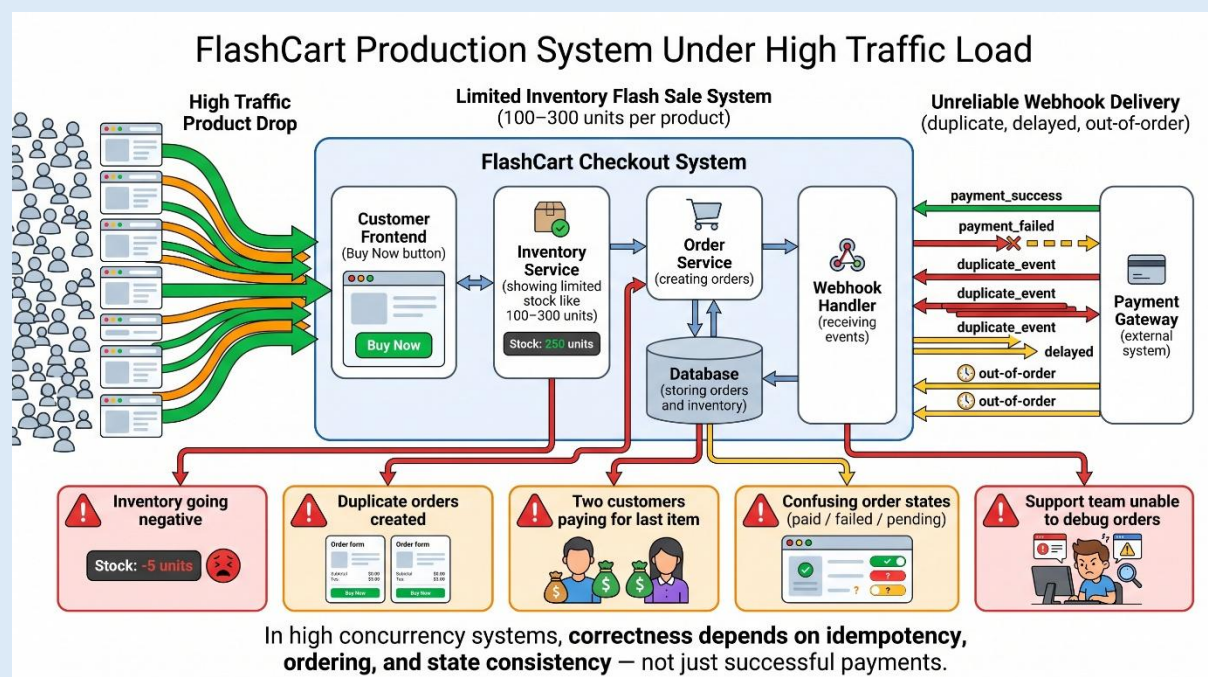
The Takeaway: Undefined requirements must be narrowed into safe iterations. It is better to block a feature than to implement undefined scope.

AI Engineering Judgment (Day 5)

The Incident: Reviewing AI-generated architecture for a flash sale.

The Action: Accepted the AI's idea to track `event_id` to prevent duplicate webhooks, but rejected its unsafe advice to use `payment_id` as the sole idempotency key (which would ignore critical refund events).

The Takeaway: AI output is an input for review, not absolute truth. State-transition boundaries must be manually verified.



Pressure Prioritization (Day 6)

The Incident: The LaunchRoom flash sale simulation broken checkout state.

The Action: Disabled payments to contain the broken checkout, auto-refunded exact duplicates, and isolated the affected SKU.

The Takeaway: Customer trust and system integrity must be protected over immediate revenue recovery, especially under time pressure.

The screenshot shows the LaunchRoom simulation interface during a crisis. The top bar indicates the current phase is "Phase 5: Final Decision Window" with a timer showing 01:24 remaining. The interface is divided into several sections:

- Startup War Room:** A list of messages from various roles (Support, Ops, Founder, Simulation Engine) discussing the crisis. Messages include: "Customers are posting screenshots. What do we communicate now?", "Clear message is helping. Tickets are still high, but customers know what will happen next.", "Decision effect applied: support tickets -80, support pressure -24, founder pressure +6, refund requests +8.", "Tickets crossed 120. Support cannot keep answering one by one.", "Rollback might worsen inventory drift if queued payment events keep landing.", "Rollback is available, but queued payment events may still land. Do we rollback, hotfix, or isolate the affected SKU?", "SKU isolation protects the rest of the drop. I can accept that if we explain it clearly.", "Decision effect applied: inventory mismatch -20, failed payments -18, support tickets -35, founder pressure +6, active users -350.", "I need a clear recommendation NOW. What do I tell the bunch team?", and "Final call: what are we containing, what are we not fixing, and what risk are we accepting?"
- Live Launch Metrics:** A table showing key performance indicators:

Metric	Value
Active users	5,545
Success rate	71%
Failed payments	168
Duplicate charges	9
Webhook lag	9m 8s
Inventory mismatch	6
Refund requests	188
Support tickets	235
Error rate	19.0%
- Decision Console:** A section for the founder to make decisions. It includes a "Final call: what are we containing, what are we not fixing, and what risk are we accepting?" prompt with a timer of 1:17s. Below this are three options: "Contain payments/orders, defer analytics", "Keep investigating before committing", and "Full stop: pause drop and reconcile". A note states: "Why this decision? What tradeoff are you accepting? We can fix the dashboards and reporting metrics tomorrow. Right now, ensuring the integrity of the payment data and stopping any further duplicate charges is the absolute top priority to protect our customers!"
- Decision Timeline:** A section showing the timeline of decisions, with a timer of 1:08. It lists "Isolate affected SKU only" and "Ops uncertainty".

2. Engineering Judgment Snapshot

Strongest Capability: Systems Reasoning & Risk Containment

Across Days 2, 5, and 6, the strongest behavioral pattern was the ability to isolate failure boundaries. Whether it was diagnosing cache poisoning, rejecting unsafe AI architectures, or containing a live flash-sale checkout failure, decisions consistently prioritized data integrity over short-term metrics.

Most Exposed Capability: Initial Debugging Speed

The weakest moment occurred at the onset of the Day 1 incident. Initial time was lost assuming backend code issues before gathering facts. The natural instinct to jump into the IDE and read code caused a delay in resolution.

3. The 30-Day Growth Prescription

The Goal: Evidence-Driven Investigation

The Habit to Break: Jumping straight into the codebase when a failure occurs.

The Habit to Build: Checking network tabs, logs, and database state as the absolute first step in any investigation.

Weekly Practice Plan

- **Week 1:** Inspect request/response headers for every bug.
- **Week 2:** Read server error logs before opening the IDE.
- **Week 3:** Verify DB/Cache state boundaries before formulating a hypothesis.
- **Week 4:** Time-box hypothesis generation to 5 minutes of log reading.

Final Commitment:

"For the next 30 days, I commit to spending the first 5 minutes of any bug investigation exclusively reading logs and network traces before I am allowed to open my IDE."